Análisis técnico: Consulta con $lookup vs optimización sin $lookup

1. Consulta original con $lookup

Este script realiza una consulta con agregación para obtener los amigos de un usuario específico ('Walker'), utilizando una operación $lookup para hacer un join con la misma colección 'user'.

Pasos del código:
- Se inicia la medición del tiempo.
- Se ejecuta una agregación con las siguientes etapas:
  1. $match: Filtra al usuario con user_id = "qVc8ODYU5SZjKXVBgXdI7w".
  2. $lookup: Realiza un join con la colección 'user', enlazando los campos 'friends' con 'user_id'.
  3. $project: Excluye el _id, incluye name y proyecta los nombres de los amigos.

- Luego se imprime el resultado del cursor y se mide el tiempo total de ejecución.

Tiempo total registrado: 4745 ms

2. Consulta optimizada sin $lookup

En este enfoque se evita el uso del costoso operador $lookup utilizando dos consultas separadas y una transformación en memoria.

Pasos del código:
- Se inicia la medición del tiempo.
- Se hace una primera consulta db.user.findOne para obtener el usuario Walker y su lista de amigos.
- Luego se hace una segunda consulta db.user.find para obtener los nombres de los amigos usando $in.
- Se transforma el cursor a un array y se aplica map para extraer los nombres.
- Finalmente se imprime el nombre del usuario y la lista de nombres de amigos.
- Se mide el tiempo de ejecución total.

Tiempo total registrado: 380 ms

3. Comparativa de rendimiento

Método	Operación principal	Tiempo total (ms)
Consulta original	$lookup en agregación	4745
Optimización sin $lookup	2 consultas + map()	380

4. Conclusión técnica

El operador $lookup es útil para realizar joins en MongoDB, pero puede tener un impacto importante en el rendimiento si se usa sobre grandes volúmenes de datos o listas extensas. Cuando sea posible, dividir la lógica en varias consultas independientes y procesar los resultados en memoria puede mejorar drásticamente la eficiencia. En este caso, se pasó de 4745 ms a solo 380 ms, una mejora de más del 90%.